This is equivalent to the -p 3000:3000 option we specified in our docker run command. If you recall, this was used to map the container's port 3000 to port 3000 on our local machine. It's needed to make our Rails app accessible from our local machine.

Launching Our App

With our docker-compose.yml in hand, Compose is now set up to manage our application. However, before we start our app, first a tiny bit of housekeeping. By default, Ruby buffers output to stdout, which doesn't play well with Compose.[2] Let's fix this by switching off Ruby's output buffering.

Add the following line to the top of your config/boot.rb file:

```
$stdout.sync = true
```

With that out of the way, we're ready to launch our app. Instead of the long docker run command, we can now use:

```
$ docker-compose up
```

Before we go through the output, let's discuss what this command does.

When you run docker-compose up, Compose makes sure that the necessary resources have been set up, creating any that are missing before launching a container for each service.

Specifically, it:

1. Creates a separate network just for the app
2. Creates any non-locally mounted volumes defined for the app (we don't have any yet—more on this in Chapter 6, Adding a Database: Postgres, on page 71)
3. Builds an image for any services with a build directive
4. Creates a container for each service
5. Launches a container per service

Pretty impressive for a single command.

If we turn our attention back to the output of the command, we can see much of this happening. First the network is created:

```
Creating network "myapp_default" with the default driver
```

2. https://github.com/sinatra/sinatra/issues/1118